

with data sets and loading data. The main classes provided by *torch.utils.data* are *Dataset* and *DataLoader*, which provide interfaces for accessing data and loading samples from a data set, respectively.

PyTorch provides not only the above mentioned standard features but also a dedicated library for NLP called *TorchText*. *TorchText* offers a number of data processing functions and several NLP data sets. It is composed of five main classes: *data*, *datasets*, *nn*, *utils*, and *vocab*. To illustrate the working, let us take a look at a simple example that demonstrates the use of *TorchText* (imported as *torchtext*) for text processing. Consider the Python script named *torch1.py* shown below (line numbers are added for ease of explanation).

```
1. import torchtext
2. from torchtext.data import get_tokenizer
3. from torchtext.data.utils import ngrams_iterator
4. tokenizer = get_tokenizer("basic_english")
5. tokens = tokenizer("Be the change you wish to see in the world")
6. print(tokens)
7. print(list(ngrams_iterator(tokens, 2)))
```

The Python script shown above finds the tokens in a string and prints all the bigrams formed from the tokens identified. But first, we need to understand what an n-gram is. An n-gram is a continuous sequence of *n* tokens from a given sample of text or speech. Bigrams are a special case of the n-gram, where *n* is 2. But why are

bigrams or n-grams critical in NLP? Well, often a lot more information can be obtained from a text or speech by considering n-grams over individual words. For example, if a word is preceded by the definite article 'the', typically this indicates that the word is being used in a specific sense rather than in a general sense. For example, consider the string '*the book*'; it refers to a specific book that has been previously mentioned whereas the word '*a book*' refers to any book without specifying a particular one.

Now, let us go through the code line by line. In our script *torch1.py*, we are considering words in the string as tokens. Line 1 imports the library *torchtext*. Lines 2 and 3 import the methods *get\_tokenizer()* and *ngrams\_iterator()*. Line 4 defines a tokenizer which splits the string into tokens after normalisation. Line 5 applies the tokenizer on the string '*Be the change you wish to see in the world*'. Line 6 prints the tokens generated. Finally, Line 7 prints all the bigrams formed from the tokens. Figure 3 shows the output of the Python script *torch1.py*. If you replace Line 7 with the line of code, '*print(list(ngrams\_iterator(tokens, 3)))*', then all the trigrams formed from the tokens will be printed. Trigrams are another special case of the n-gram, where *n* is 3. PyTorch is a general machine learning framework with a lot of features for NLP. However, there are also libraries designed specifically for NLP. Next, we discuss one such library called natural language toolkit (NLTK).

## NLP using NLTK

NLTK is a symbolic and statistical NLP Python library. It is free and open source software licensed under Apache License version 2.0. NLTK can be used for text processing, tokenisation, part-of-speech tagging, sentiment analysis, etc. As usual, the installation of NLTK can be done easily with the help of Anaconda Navigator. Now, let us see how a Python script similar to *torch1.py* can be written using NLTK. Consider the program *nlk1.py* shown below, which finds the tokens in a string and prints all the bigrams formed from the tokens identified (line numbers are added for ease of explanation).

```
1. import nltk
2. from nltk.tokenize import word_tokenize
3. tokens = nltk.word_tokenize("Be the change you wish to see in the world")
4. print(tokens)
5. print(list(nltk.ngrams(tokens, 2)))
```

Now let us go through the code line by line. Line 1 imports the *nltk* library. Line 2 imports the method *word\_tokenize()*. Line 3 applies the function *word\_tokenize()* on the string '*Be the change you wish to see in the world*'. Line 4 prints the generated tokens, and finally, Line 5 prints all the bigrams formed from the tokens. Figure 4 shows the output of the Python script *nlk1.py*. If you replace line 5 with the line of code '*print(list(nltk.ngrams(tokens, 3)))*', then all the trigrams formed from the tokens will be printed. Notice that NLTK also provides a function called *bigrams()* which will generate all the bigrams from a set of tokens.

```
# python nlk1.py
['Be', 'the', 'change', 'you', 'wish', 'to', 'see', 'in', 'the', 'world']
[('Be', 'the'), ('the', 'change'), ('change', 'you'), ('you', 'wish'), ('wish', 'to'), ('to', 'see'), ('see', 'in'), ('in', 'the'), ('the', 'world')]
```

Figure 4: Finding bigrams with NLTK